CyberShield AI - Product Requirements Document

Version: 1.1
Product type: Local cybersecurity awareness and screening web application
Audience: Students, general users, evaluators, and maintainers
Status: Functional local MVP

1. Product Summary

CyberShield AI helps a user inspect suspicious text messages, URLs, and files and understand which signals caused the result. The application combines a trained machine-learning SMS classifier, transparent URL heuristics, and file metadata/signature checks. It runs locally and stores account and scan records in SQLite.

CyberShield AI is an educational screening tool. It is not antivirus software, does not sandbox files, and cannot prove that a message, website, or file is safe.

2. Problem and Opportunity

Non-technical users often receive suspicious messages, links, and downloads but do not know which clues to examine. A simple verdict without explanation does little to teach safer habits. CyberShield AI provides an accessible interface that makes its indicators visible and gives practical next-step recommendations.

3. Goals

- Provide an end-to-end local application with a React/TypeScript frontend and FastAPI backend.
- Explain each result with indicators, a score, the detection method, and a recommendation.
- Use a real trained model and report measured held-out evaluation results.
- Keep submitted scan data and account records in the local SQLite database; do not fetch submitted URLs or execute uploaded files.
- Require a local account before accessing the dashboard and keep scan history isolated by account.
- Present limitations clearly and avoid claiming guaranteed safety.

4. Non-Goals

- Antivirus, malware sandboxing, or inspection of executable behavior.
- Live URL visits, threat-intelligence feeds, malware reputation lookups, or paid APIs.
- Cloud hosting, multi-tenant service operation, or production-grade identity management.
- Multilingual or transformer-based message classification.
- PDF scan-report export in the current MVP.

5. Target Users

User	Need
Student/general user	Check an unexpected message, link, or file and understand warning signs
Evaluator	Review a working ML pipeline, measured metrics, and visible limitations
Developer/teacher	Run the local project and inspect its code and tests

6. Features and Priorities

P0 - Core MVP

ID	Feature	Acceptance criteria
F1	Local signup and sign-in	User can create an account and sign in; passwords are stored as salted hashes, not plain text
F2	Session and access control	Dashboard and scan/history APIs require a valid session; logout revokes that session
F3	Message spam scanner	Trained TF-IDF + Logistic Regression returns label, spam probability, confidence, contributing terms, hints, and recommendations
F4	URL screening	Rules return a 0-100 score, Low/Medium/High risk verdict, and triggered explanations; URL is never fetched
F5	File screening	File is read as bytes only; result includes SHA-256, size, type/signature clues, and risk flags
F6	Explainable results	Result identifies its method and shows indicators and recommendations without asserting guaranteed safety
F7	Scan history and dashboard	Authenticated user can see their own history and statistics; another account cannot read or delete it
F8	Input handling	Invalid text/URL and oversized file uploads return clear client errors

P1 - Supporting MVP pages

ID	Feature	Acceptance criteria
F9	Model evaluation	Page reads accuracy, precision, recall, F1, split details, and confusion matrix from generated metrics
F10	Awareness content	Page provides concise safe-message, link, credential, and file practices
F11	Responsive interface	Login, dashboard, scanner, history, evaluation, and awareness pages adapt to desktop and phone widths
F12	Sample inputs	Scanner provides one-click demo inputs for message and URL flows

Deferred / Future

- PDF report export.
- History filtering controls in the UI and chart visualizations.
- More comprehensive file-format detection, antivirus integration, and external reputation checks.
- Password reset, email verification, MFA, and hardened internet-facing account management.
- Deployable cloud/multi-user configuration.

7. Product Workflows

First visit and authentication

1. The frontend asks the API whether a valid session exists.
2. A signed-out user sees the sign-in or create-account screen.
3. Signup validates name, email, and a password of at least 10 characters.
4. The backend hashes the password with a unique salt, saves the account locally, and issues an expiring HttpOnly session cookie.
5. The frontend shows the dashboard only after session verification. Sign-out revokes the server-side session.

Scan workflow

1. User selects Message, Link, or File inspection.
2. The frontend validates basic input length and calls the corresponding authenticated API endpoint.
3. The backend runs the model or analyzer, builds an explainable result, and saves it under the signed-in user's ID.
4. The result is displayed and dashboard/history data refreshes.

8. Detection Design

Message classification

- Dataset: UCI SMS Spam Collection; 5,572 records in the bundled training run.
- Features: English-stop-word-filtered TF-IDF with unigram/bigram terms.
- Classifier: class-weighted Logistic Regression.
- Evaluation: deterministic stratified 80/20 train/test split.
- Explanation: highest positive spam-weight terms present in the message plus separately labelled heuristic hints.

Measured metrics for the bundled model: accuracy 97.49%, precision 89.54%, recall 91.95%, and F1 90.73% on 1,115 held-out messages. These are dataset-specific results, not a guarantee of current real-world performance.

URL analysis

Heuristics check IP hosts, URL length, subdomain count, `@`, hyphens, HTTPS, sensitive-action keywords, known shorteners, punycode, selected suspicious top-level domains, and known brand names outside expected domains. The scanner never makes a network request to the submitted URL.

File analysis

The service enforces a 10 MB limit, reads content as bytes in memory, calculates SHA-256, guesses MIME type, checks a small set of magic-byte signatures, and flags selected risky/double extensions and mismatches. It never executes, opens, or renders the uploaded file, and stores no file bytes. This does not inspect file contents for malware.

9. Success Measures

- All core P0 workflows work end to end locally.
- Scanner results are produced by the real model/rules/metadata checks, not hard-coded demo verdicts.
- Account-specific history and dashboard stats remain isolated.
- Model evaluation is reproducible from the training script and dataset.
- A short local demonstration can show signup, scans, history, and evaluation without external service dependencies (after install).

10. Constraints and Risks

Risk	Response
One-day/student project scope	Keep the current implementation local and prioritize reliable core flows
Old English-only SMS corpus	Show dataset and evaluation caveat prominently
URL heuristics may be wrong	Show individual signals and explain rule-based method
Metadata-only file checks are limited	Never label a file safe; state that no malware scan is performed
Local account flow is not production IAM	Keep deployment local; document missing recovery, verification, rate limiting, and production hardening
Model binary compatibility	Retrain with the included script if the installed scikit-learn version cannot load the bundled artifact

11. Current Project Layout

```
backend/
  app/main.py                 FastAPI routes, auth, analysis, and SQLite access
  ml/train_spam_model.py      Model training and evaluation
  ml/models/                  Bundled model artifact and metrics
  tests/test_api.py           Auth and scanner API tests
  requirements.txt            Python dependencies
frontend/
  src/App.tsx                 Auth gate, dashboard, scanners, and app pages
  src/AuthScreen.tsx          Login and signup interface
  src/*.css                   Theme and responsive styles
  package.json                JavaScript dependencies and commands
```

12. Setup and Run

From the project root in PowerShell, create and install a Python environment if needed:

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
```

Install frontend dependencies:

```
cd frontend
npm install
cd ..
```

Run the API in one terminal:

```
cd backend
..\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Run the UI in another terminal:

```
cd frontend
npm run dev -- --port 5173
```

Open `http://127.0.0.1:5173/` and create a local account. The ZIP includes the trained model and metrics. To retrain, put the UCI `SMSSpamCollection` file at `backend/ml/data/SMSSpamCollection`, then run `..\.venv\Scripts\python.exe ml\train_spam_model.py` from `backend`.

13. Test Commands

```
cd backend
..\.venv\Scripts\python.exe -m pytest -q
cd ..\frontend
npm run build
```